

回滚网络同步中输入预测的建模与仿真：格斗游戏中的延迟、确定性与重演代价权衡

Modeling and Simulation of Input Prediction in Rollback Netcode: The Latency, Determinism and Resimulation Tradeoff in Fighting Games

yamwelike, copycat666^{1,2*}, doro^{1,2}

1. EigenLife 机器学习实验室，东北电力大学，吉林；2. 《电子游戏科学与技术》编辑委员会，吉林，
submission.joest.cn@outlook.com

*: Corresponding Author

摘要：格斗游戏对输入延迟极为敏感。回滚式网络同步以先预测后修正的方式把本地输入延迟降到零，代价是额外的重演计算与画面翻案。本文用可复现的确定性仿真栈刻画该权衡：以 Q16.16 定点数实现 60 Hz 双人格斗游戏微内核，以逐帧冗余链路模型驱动 3 种预测策略并与延迟式对照。结果表明：（1）浮点同步极为脆弱：把平方和收缩成单次舍入的 FMA 即在第 871 帧产生静默分歧（偏差 7.6 px），float32 第 1 帧即分歧，而定点在语句重写后仍逐位一致；（2）回滚把本地输入延迟由 3 至 8 帧降到 0 帧，代价是 17.1% 至 69.0% 的额外模拟开销与 0.4 至 31.4 次/分钟的可见翻案；（3）猜错率随延迟由 15.9% 升到 33.4%，但回滚触发率稳定在 14.1% 至 16.0%；（4）预测增益上限很小，方向肢已被保持策略封顶（1 帧前瞻 84.3%），三种策略 CPU 代价相差不到 2 个百分点；（5）3 重冗余把 5% 丢包的影响压到 0.5 个百分点内，缓冲窗口取 10 帧即可消除较差网络下的停顿；全部回滚时间线与真值逐位一致。

关键词：回滚网络同步；输入预测；确定性仿真；定点数运算；延迟隐藏；格斗游戏

Abstract. Fighting games are extremely sensitive to input latency. Rollback netcode hides latency by predicting remote inputs and correcting later, driving the local input delay to zero at the cost of extra resimulation and visible corrections. This paper builds a reproducible deterministic simulation stack to quantify that tradeoff. A 60 Hz two-player fighting-game kernel is implemented in Q16.16 fixed point, driven by a frame-level redundant link model with three input predictors, and compared against delay-based netcode. Results: (1) floating-point builds desynchronize silently—contracting the sum of two squared terms into a single-rounding FMA is enough to diverge at frame 871 with a 7.6 px position error, while float32 diverges at frame 1, whereas the fixed-point build stays bit-identical even after statement reordering; (2) rollback removes the 3–8 frame local input delay of delay-based netcode at the cost of 17.1%–69.0% extra simulation load and 0.4–31.4 visible corrections per minute; (3) the guess error rate grows from 15.9% to 33.4% with latency, yet the rate that actually triggers a rollback stays at 14.1%–16.0%, the difference being absorbed by incidental resimulation; (4) the headroom of better prediction is small—the direction limb is already capped by the hold strategy (84.3% at a one-frame horizon) and the three predictors differ by less than 2 percentage points in CPU cost; (5) three-fold redundancy keeps the effect of 5% packet loss within 0.5 percentage points (only one unrecoverable frame in 200 s), and a 10-frame buffer window eliminates stalls under RTT 150 ms with 2-frame jitter and 3% loss. All rollback timelines are bit-identical to ground truth, validating the implementation.

Keywords: Rollback Netcode; Input Prediction; Deterministic Simulation; Fixed-Point Arithmetic; Latency Hiding; Fighting Games

一、引言

1.1. 研究背景

格斗游戏是电子竞技中对时间最苛刻的一类。主流作品以 60 Hz 定步长推进游戏逻辑，一帧只有 16.67 ms，而一次攻击判定通常只有 2~3 帧宽。这意味着玩家按下按键后，若画面再晚 3 帧响应，连段就会失败、确认就会落空。因此格斗游戏的网络同步方案长期被一个问题统治：**如何在不可靠的公网上，让远端玩家的输入以近似零延迟的方式参与本地判定。**

工程上有三条路线。第一条是**延迟式同步**（delay-based）：双方把本地输入缓冲 d 帧后再执行，保证执行第 t 帧时双方的第 t 帧输入都已到达。它的确定性最好，但代价是把 d 帧（通常取单向延迟的高分位数）直接加到玩家的手感延迟上。第二条是**状态插值**：接收方外推远端角色的状态，适合射击与 MMO，却无法用于”判定必

须双方一致”的招式对拼。第三条是**回滚式同步**（rollback netcode）：本地立刻用预测的远端输入推进，待真值到达后，若预测有误就回滚到出错帧、用已知信息重新推进到当前帧。玩家的本地输入延迟因此为 0，代价是额外的重演计算与偶尔可见的画面翻案。

回滚式同步已在商业格斗游戏中大规模落地，但公开文献中**定量**刻画其收益与代价的工作仍然稀少：收益（延迟消除）容易被感知，代价（CPU 开销、画面翻案频率、停顿风险）却高度依赖实现细节与网络条件，缺乏可复现的测量口径。本刊此前的分布式计算架构综述 [1] 指出，低延迟是游戏系统架构的核心矛盾之一，但没有展开到“同步协议内部”的取舍。

1.2. 本文工作与贡献

本文不复述工程经验，而是建立一套**端到端可复现的确定性仿真栈**，把回滚同步的三个关键量——延迟、确定性、重演代价——放到同一组实验里测量。贡献如下：

(1) **一个确定性游戏微内核与数值格式对照实验**。用 Q16.16 定点数实现 60 Hz 双人格斗游戏内核，并构造 5 个数值执行变体（float64、float64+FMA 收缩、float32、Q16.16、Q16.16 语句重写），量化不同数值格式下的同步分歧行为（第 5.1. 节）。

(2) **回滚客户端与链路模型**。实现带状态缓冲、冗余重发与回滚重算的客户端算法，并把单向延迟、抖动、丢包三个网络自由度显式参数化（第 3 节）。

(3) **收益一代价的定量刻画**。给出延迟式与回滚式在本地输入延迟上的差（3~8 帧 vs 0 帧），以及回滚所付出的额外模拟开销（17.1%~69.0%）、可见翻案频率（0.4~31.4 次/分钟）与停顿风险随 RTT、抖动、丢包的变化规律（第 5 节）。

(4) **预测增益边界的分析**。通过把预测质量与网络解耦的“预测前瞻—准确率”实验，说明为什么更好的输入预测器对回滚代价的改善上限很小（第 5.2. 节）。

(5) **正确性验证**。所有回滚时间线与真值时间线逐位比较，分歧帧数为 0（第 5.6. 节），为上述度量提供可信载体。

二、相关工作

2.1. 锁步确定性与定点数运算

确定性锁步（deterministic lockstep）是实时战略游戏时代确立的方案，其基本要求是“相同的输入序列在任意机器上产生逐位相同的状态序列”[2]。游戏物理的数值稳定性在本刊亦有讨论 [3]。浮点算术的不可移植性早已被系统研究：IEEE-754[4] 规定了单次运算的舍入，却允许编译器做 FMA 融合（-ffp-contract=fast）、允许库函数使用不同精度的实现，也允许在寄存器中保留扩展精度 [5, 6, 7]。因此“同一份源代码在不同平台上给出不同结果”不是罕见事故，而是默认行为。整数定点运算没有这类自由度：运算精确、结合律与交换律成立，只要显式阻断 FMA 融合即可获得位级可复现性。

2.2. 回滚式同步

回滚同步的工程范式由 GGPO 一类库确立 [8]：发送端在数据包中冗余携带最近若干帧的输入以抗丢包；接收端保存状态快照，在发现预测错误时回滚重算。客户端-服务器架构下的延迟补偿（lag compensation）是另一条路线的经典工作 [9]，其思想是让服务器“回到过去”判定射击命中。面向开发者的系统性介绍可见文献 [10, 11]。

2.3. 输入预测

远端输入预测在形式上是序列预测问题。最朴素的做法是“保持”（重复上一输入）。更强的做法是上下文建模：一阶/二阶马尔可夫链、以及自适应上下文混合（PPM）[12]；平滑与退避的统计基础见文献 [13]。格斗游戏输入的特殊性在于：方向肢高度惰性（玩家长时间按住某个方向），按键肢高度稀疏（攻击键只在少数帧按下），而“搓招”会在 2~6 帧内制造一串方向切换。这三者叠加使得预测器在不同前瞻长度上的表现差异很大，第 5.2. 节将定量给出这一边界。

2.4. 与已有工作的区别

与本刊已发表工作相比：分布式计算架构综述 [1] 关注云、边缘与网络基础设施层面的低延迟，本文聚焦同步协议内部的输入预测与重演代价；实时物理的数值积分工作 [3] 关注能量守恒，本文关注数值格式对多端一致性的影响。与通用网络编程文献 [2, 9, 8, 10] 相比，本文的差异在于：不给出实现建议，而是给出一个可复现的

仿真栈与一组定量测量口径（猜错率、回滚触发率、回滚深度分布、CPU 相对开销、可见翻案频率、停顿帧数），并明确区分“预测错误”与“真正触发回滚”这两个常被混为一谈的量。

三、仿真模型

3.1. 确定性游戏微内核

游戏是一维场地（长 320 px）上的双人对抗，逻辑帧率 60 Hz。每帧的处理顺序被严格固定为四步：(1) 由双方位置确定朝向；(2) 各自依据输入更新自身状态（受击硬直 → 出招 → 移动/防御）；(3) 推进位置、裁剪到场地边界、分离到最小间距；(4) 结算出招命中——双方使用“移动后、结算前”的同一份状态，因此允许相杀。

出招时间轴为 3 帧起手、2 帧判定、4 帧收招，共 9 帧；命中造成 8 点伤害、12 帧硬直与击退，被防御时造成 1 点削血与 4 帧防御硬直。状态为

$$s = (\mathbf{p}_1, \mathbf{p}_2), \quad \mathbf{p}_i = (x_i, v_i, h_i, a_i, d_i, s_i, g_i), \quad (1)$$

全部为整数（位置与速度用 Q16.16，其余为小型计数）。状态因此可以 O(1) 时间“快照”（元组引用），这正是回滚可以在数十帧范围内实时重算的前提。

数值格式上，定点乘法一律写成“先乘后移位”

$$\text{mul}(a, b) = (a \cdot b) \gg 16, \quad (2)$$

移位把整个表达式拆成精确的整数乘与精确的整数移位两步，编译器无法把它融合成单次舍入的 FMA；平方根用整数算法 `math.isqrt`，结果唯一、不依赖 `libm`。

3.2. 链路模型

每帧发送一个输入包，包内冗余携带最近 $R = 3$ 帧的输入（GGPO 一类实现抗丢包的基本手段）。单向延迟 $\text{owd}[t]$ 为基础延迟叠加逐帧独立高斯抖动，取整到帧且不小于 1；丢包 $\text{lost}[t]$ 为逐帧独立伯努利事件。于是“本地第 u 帧时刻已确认远端第 t 帧输入”的最早时刻为

$$\text{arr}(t) = \min\{s + \text{owd}[s] \mid s \in [t, t + R - 1], \neg \text{lost}[s]\}. \quad (3)$$

若窗口内 R 份副本全部丢失，则该帧输入在本会话内不可恢复。式 (3) 取最小而非平均，带来一个非直观后果：抖动增大反而会让有效延迟下降（见第 5.5 节）。

3.3. 回滚客户端

本地按 60 Hz 匀速推进，对尚未确认的远端输入使用预测值；当某帧真值到达且与当前所用值不符时，回滚到该帧并用当前已知信息重算到当前帧。伪代码见算法 1，时序示意图 1。

算法 1 中有一个容易被忽略的细节：被回滚区间 $[r, u)$ 内的帧会被重新预测，此时预测器已掌握更新的事实。因此一帧可能先后被两次预测——第一次是它被首次推进时（“首次预测”），第二次是被后续回滚顺带重算时。这两次预测的正确性对应两个不同的量，第 3.6 节将分别定义。

3.4. 输入轨迹模型

逐帧级的格斗游戏人类输入数据集公开资源稀少，直接套用单一样本会把结论绑死在特定对局上。本文因此采用参数化生成模型，只刻画四条公认的统计规律，并把自由度显式暴露成可扫描参数：(1) **方向保持性**——玩家绝大多数时间在“按住”某个方向；(2) **意图分层**——中立/进攻/防御/移动四类意图的驻留时间尺度差别很大；(3) **按键稀疏**——攻击键只在极少数帧被按下且总与某方向同时按住；(4) **招式套路化**——高阶操作是固定的短方向序列（半圆 236、反半圆 214、前后冲刺），同一玩家招式集合小、重复率高，但每个方向段只有 1~3 帧。

为让对局不退化（双方各自缩到场地两端、攻击打不到人），生成器采用距离驱动的闭环策略：每到一个新意图段的起点，按当前双方间距选择意图（远 → 逼近，中 → 试探，近 → 压制/出招），再逐帧展开。生成结果是两条固定的逐帧输入序列，后续所有同步实验只是回放这两条序列，与网络的随机性完全解耦。主实验配置下（200 s）的轨迹特征为：攻击键按下帧占 3.6%~4.0%，防御帧占 15.1%~20.5%，中立帧占 5.0%~5.6%，方向段平均长度 9.8~10.3 帧，方向切换率约 10%/帧，命中事件 160~193 次。作为参照，第 5.5 节的节奏扫描把方向段平均长度从 6 帧扫到 28 帧、方向切换率从 14.3% 扫到 7.8%，覆盖了从“高压连段”到“慢节奏试探”的范围。

Algorithm 1 回滚客户端的逐帧处理

Require: 本地输入 $i_L[\cdot]$, 链路参数, 缓冲窗口 $D_{\max}$, 预测器 $\mathcal{P}$

```

1:  $\mathcal{S}[0] \leftarrow s_0$  (状态缓冲),  $\mathcal{K} \leftarrow \emptyset$  (已确认远端输入),  $\text{used}[\cdot] \leftarrow \text{空}$ 
2: for  $u = 0$  to  $N - 1$  do
3:    $r \leftarrow \text{nil}$ 
4:   for all 在本地帧  $u$  到达的远端帧  $t$  do
5:      $\mathcal{K}[t] \leftarrow u_{\text{real}}[t]$ ;  $\mathcal{P}.\text{observe}(t, \mathcal{K}[t])$ 
6:     if  $\text{used}[t] \neq \mathcal{K}[t]$  then
7:        $r \leftarrow \min(r, t)$ 
8:     end if
9:   end for
10:  if  $r \neq \text{nil}$  then
11:    for  $k = r$  to  $u - 1$  do
12:       $\text{used}[k] \leftarrow \mathcal{K}[k]$  (若已知, 否则  $\mathcal{P}.\text{predict}(k)$ )
13:       $\mathcal{S}[k + 1] \leftarrow \text{step}(\mathcal{S}[k], i_L[k], \text{used}[k])$ 
14:    end for
15:    记录回滚深度  $D \leftarrow u - r$ ; 若  $D > D_{\max}$  记一次停顿
16:  end if
17:   $\text{used}[u] \leftarrow \mathcal{K}[u]$  (若已知, 否则  $\mathcal{P}.\text{predict}(u)$ )
18:   $\mathcal{S}[u + 1] \leftarrow \text{step}(\mathcal{S}[u], i_L[u], \text{used}[u])$ 
19: end for

```

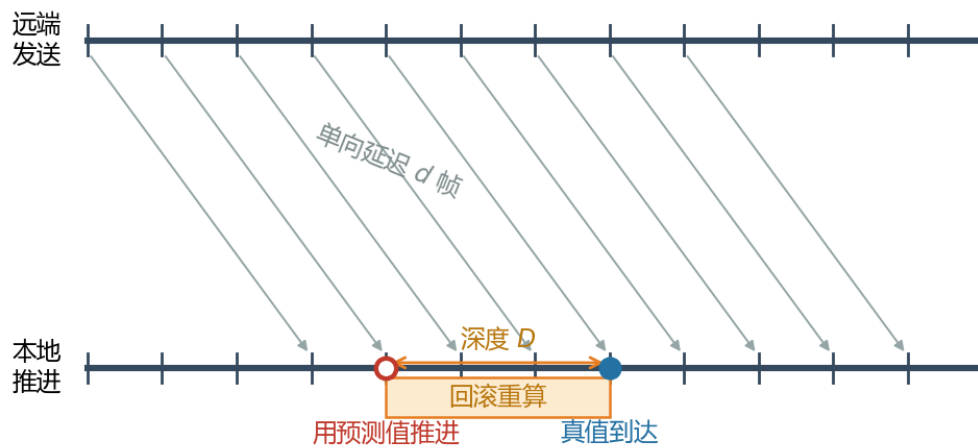


图 1: 回滚同步时序示意: 本地帧 u 先用预测值推进, 单向延迟 d 帧后真值到达, 若预测有误则回滚 D 帧并重算

3.5. 输入预测器

表 1: 三种输入预测策略

记法	策略
S0	保持 : 直接重复最近一帧已确认的输入（工程上最常用的默认策略）。
S1	一阶马尔可夫 : 把输入拆成方向肢（无/左/右）与按键肢（无/攻/防/攻防），对相邻帧的转移计数加拉普拉斯平滑后分别取条件众数。
S2	二阶退避 + 惯性打分 : 两个肢都用二阶上下文、以一阶退避；方向肢额外叠加“运行长度先验”——方向已保持 L 帧时给“继续保持”加分，分值随 L 线性增长。
S3	完美预测 （仅作理论上界，给出回滚代价的下界）。

S2 的打分函数为

$$\text{score}(d) = \beta c_2[(p_2, p_1, d)] + c_1[(p_1, d)] + \lambda(L + 1)[d = p_1] + \alpha, \quad (4)$$

其中 p_1, p_2 是上两帧的方向肢， L 是当前方向段的已持续帧数， $\beta = 0.7$ ， $\lambda = 0.35$ ， $\alpha = 0.5$ 。全部预测器都按**帧号索引历史**：只允许读取帧号小于待预测帧的输入，因此（a）严格因果，回滚重算时不会偷看未来；（b）对乱序到达免疫——抖动大时第 $t + 1$ 帧可能先于第 t 帧到达，按帧号存储才能正确回填。

3.6. 度量定义

设统计窗口为 N 帧，第 t 帧远端输入的真值为 $u(t)$ ，“首次预测”值为 $\hat{u}_0(t)$ ，“真值到达时所用”值为 $\hat{u}_1(t)$ 。定义：

$$\rho_{\text{guess}} = \frac{1}{N} |\{t : \hat{u}_0(t) \neq u(t)\}| \quad (\text{猜错率: 客户端第一次猜就错的比例}), \quad (5)$$

$$\rho_{\text{trig}} = \frac{1}{N} |\{t : \hat{u}_1(t) \neq u(t)\}| \quad (\text{回滚触发率: 真值到达时仍然错的比例}), \quad (6)$$

$$\rho_{\text{self}} = \rho_{\text{guess}} - \rho_{\text{trig}} \quad (\text{顺带纠正率}), \quad (7)$$

$$\bar{D} = \frac{1}{|\mathcal{E}|} \sum_{e \in \mathcal{E}} D_e \quad (\text{平均回滚深度}), \quad (8)$$

$$O = \frac{1}{N} \sum_{e \in \mathcal{E}} D_e \quad (\text{CPU 相对开销: 额外重算帧数 / 实推进帧数}), \quad (9)$$

$$V = \frac{60 \cdot 60}{N} |\{e \in \mathcal{E} : \text{重算区间内命中事件发生改变}\}| \quad (\text{可见翻案, 次/分钟}). \quad (10)$$

其中 $\mathcal{E}$ 是回滚事件集合， D_e 为该次回滚的深度。 O 的含义是“每推进一帧平均要多重算 O 帧”，因此在单帧步进耗时相近的实现里，它直接等于 CPU 的额外占用比例；本文同时实测了单帧步进耗时与状态快照的相对开销（第五节）。 V 度量的是玩家真正能看见的翻案：被重算的区间里若“谁打中了谁”发生改变，画面上就会出现“这一刀刚才明明打中了”的修正感。

此外定义**停顿**： $D_e > D_{\max}$ 时客户端必须停顿等待（状态缓冲用尽），本文记停顿次数与累计停顿帧数 $\sum_e \max(0, D_e - D_{\max})$ 。

四、实验设置

除特别说明，实验参数如表 2 所示。全部实验在 Python 3.8 下运行，单帧状态步进实测 $1.09 \mu\text{s}$ ，状态快照（元组引用）仅为步进耗时的 0.93%，因此 O 可以近似直接读作 CPU 额外占用比例。

表 2: 仿真参数

参数	取值	参数	取值
逻辑帧率	60 Hz	统计窗口 N	12000 帧 (200 s)
场地长度	320 px	冗余度 R	3
出招时间轴	3+2+4 帧	缓冲窗口 $D_{\max}$	7 帧
伤害 / 削血	8 / 1	单向延迟	RTT/2/16.67 帧
硬直 / 防御硬直	12 / 4 帧	抖动 σ	1 帧
方向段平均长度	14 帧	丢包率	2%
每秒方向切换率	10.0%	随机种子	3 组

五、实验结果

5.1. 数值格式与同步确定性

在”双球抛体—壁面反弹—软碰撞”微基准上运行 3600 帧（含欧氏距离归一化，引入 sqrt 与除法），比较 5 个数值执行变体，结果见表 3 与图 2。

表 3: 五种数值执行变体的同步分歧（3600 帧，相对本族参考实现）

变体	首次分歧帧	最大位置偏差 / px	末帧偏差 / px	碰撞判定错位帧
float64（参考）	无	0	0	0
float64 + FMA 收缩	871	7.63	1.05	0
float32	1	283.55	100.66	1004
Q16.16（参考）	无	0	0	0
Q16.16 + 语句重写	无	0	0	0

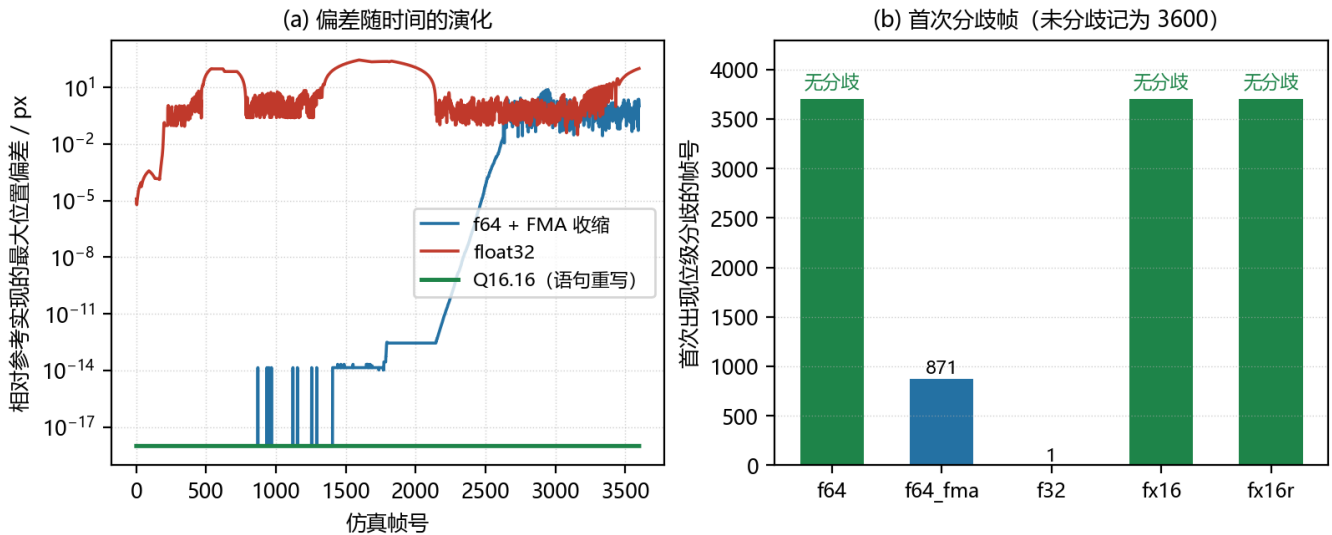


图 2: 数值格式对同步确定性的影响: (a) 偏差随时间的演化（纵轴对数）; (b) 首次出现位级分歧的帧号

三点结论值得强调。第一，**FMA 收缩足以造成静默分歧**：float64 参考实现在所有语句照写时完全确定，但只要把

$$d^2 = dx \cdot dx + dy \cdot dy \quad (11)$$

收缩成单次舍入的 $\text{fma}(dx, dx, dy \cdot dy)$ ——这正是编译器在 `-ffp-contract=fast` 下的默认行为——分歧就在第 871 帧（约 14.5 s 对局）出现。两个平方项量级相当，式 (11) 恰好是 FMA 会改变舍入结果的经典场景；由于 d^2 随后要参与”是否发生碰撞”的阈值比较，舍入差异会直接翻转布尔判定。本文 3600 帧内未观测到碰撞判定错位（偏差尚小），但这只是时间问题：偏差一旦跨越阈值就会跳变。

第二，**float32 是灾难性的**：第 1 帧即出现分歧，最大位置偏差达 283.55 px（超过场地长度），3600 帧中有 1004 帧（27.9%）的碰撞判定与参考实现不一致。任何跨平台同步的游戏都不应让浮点宽度依赖平台。

第三，**定点实现具有结构性优势**：Q16.16 实现不仅在参考写法下逐位一致，把语句顺序、结合方式、常量预计算全部重写后仍逐位一致（表 3 最后一行）。原因很直接：整数运算精确，交换律与结合律成立，而式 (2) 的“先乘后移位”写法切断了 FMA 融合的可能。

需要说明的是，定点与浮点是两套不同的数值规范，二者相对位置偏差可达 227.2 px——即“换数值格式”等价于“换了一个游戏”，这会让既有录像与存档失效。确定性要求的是“同一款游戏的所有端一致”，而非“实现与浮点参考一致”。

5.2. 预测器的能力边界

把预测质量与网络解耦：逐帧喂入真值，再让预测器在“只看到第 t 帧为止的输入”的条件下预测第 $t + h$ 帧，得到“预测前瞻—准确率”曲线（图 3，表 4）。当单向延迟为 h 帧时，客户端面对的正是这个问题。

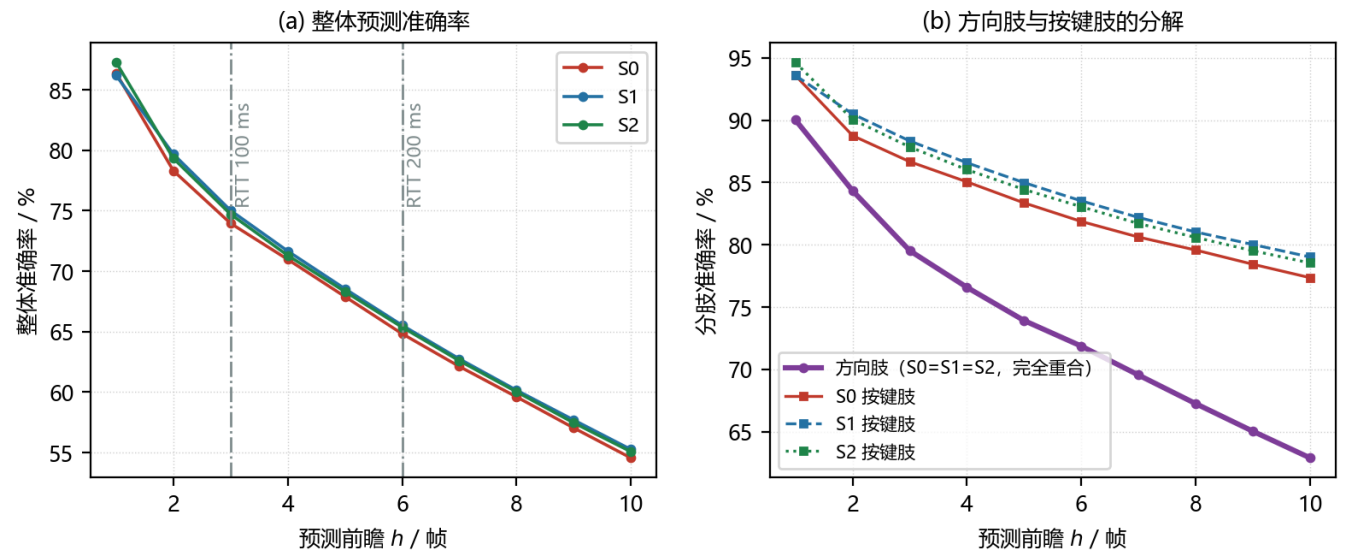


图 3: 预测准确率随预测前瞻的变化。(a) 整体准确率，虚线标出 RTT 100 ms 与 200 ms 对应的实际工作点；(b) 分枝分解——方向肢上三种策略完全重合

表 4: 预测准确率与预测前瞻（单位：%）

肢	策略	$h = 1$	$h = 2$	$h = 3$	$h = 5$	$h = 10$
整体	S0	78.30	73.96	70.97	64.83	52.25
	S1	79.67	75.02	71.66	65.53	52.93
	S2	79.33	74.71	71.30	65.37	52.78
方向肢	S0	84.33	79.52	76.58	71.87	60.84
	S1	84.33	79.52	76.58	71.87	60.84
	S2	84.33	79.52	76.58	71.87	60.84
按键肢	S0	88.76	86.67	85.05	81.87	76.42
	S1	90.50	88.33	86.58	83.52	78.08
	S2	90.05	87.85	86.03	83.05	77.63

最重要的观测是**方向肢上三种策略完全重合**。原因并不难理解：方向过程近似“长段保持 + 偶发短切换”，在任意上下文下，下一帧最可能的取值始终是“不切换”，因此一阶、二阶马尔可夫的极大似然解退化为“保持”，与 S0 完全相同。换句话说，**方向肢已经被“保持”策略封顶**：1 帧前瞻 84.33%，10 帧前瞻 60.84%，这条曲线就是该输入模型下方向预测的天花板。

可提升空间全部落在按键肢。S1 的转移统计把按键肢从 88.76% 提升到 90.50% ($h = 1$)、从 81.87% 提升到 83.52% ($h = 5$)，整体准确率随之从 78.30% 提升到 79.67%。有意思的是 S2 **略逊于** S1：式 (4) 的惯性先验对方

向肢无害（二者解相同），但对按键肢有害——攻击键是 1~2 帧的短脉冲，“按得越久越可能继续按”的先验会诱导预测器多预测一帧攻击。

因此，预测精度的提升是结构性受限的：整体准确率的最佳改进只有 1.4 个百分点（ $h = 1$ ），而这需要引入一个比自己训练还复杂的在线模型。这个结论为下一节的“预测器对回滚代价影响很小”提供了直接解释。

5.3. 收益：本地输入延迟的消除

表 5 给出延迟式与回滚式的手感延迟对照。延迟式取 $D = \lceil Q_{0.99}(\text{owd}) \rceil$ 帧作为输入缓冲（按 99 分位取值以免抖动造成丢输入），其本地输入延迟即为 D ，端到端响应为 $D + \text{owd}$ 帧。回滚式的本地输入延迟恒为 0，端到端响应仅为单向延迟。

表 5: 延迟式与回滚式的手感延迟对照（单位：帧）

RTT / ms	30	60	90	120	150	200
平均单向延迟 owd	1.34	1.91	2.71	3.60	4.50	6.00
延迟式输入缓冲 $D (p_{99})$	3	4	5	6	7	8
延迟式端到端响应	4.34	5.91	7.71	9.60	11.50	14.00
回滚式本地输入延迟	0	0	0	0	0	0
回滚式端到端响应	1.34	1.91	2.71	3.60	4.50	6.00

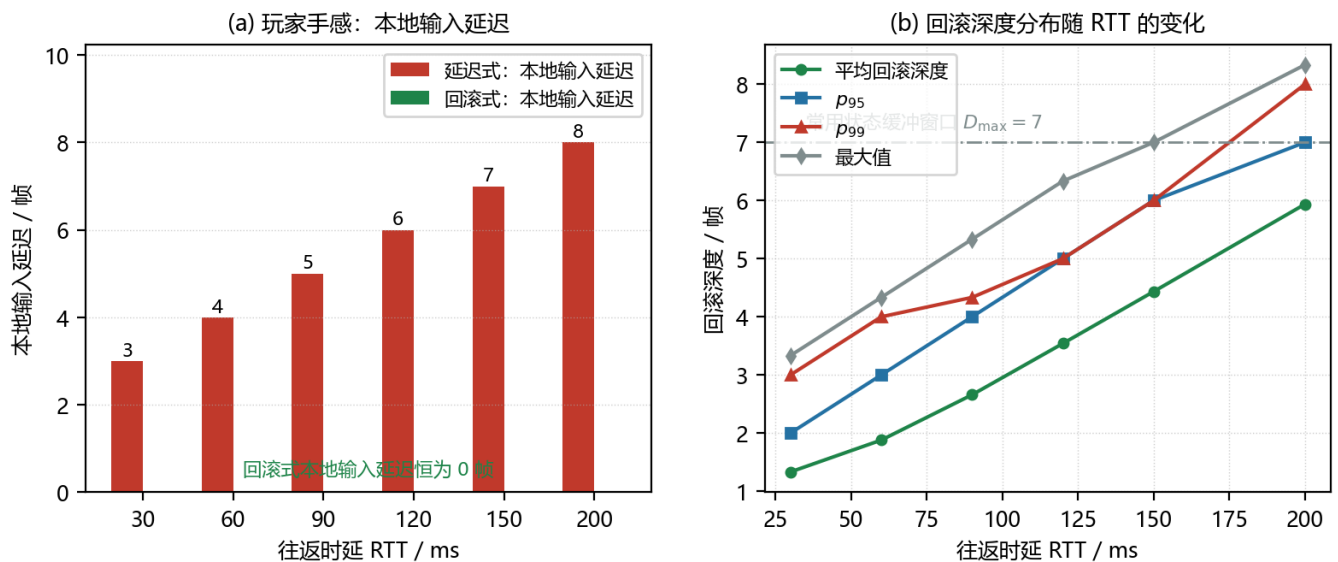


图 4: (a) 本地输入延迟：延迟式为 3~8 帧，回滚式恒为 0；(b) 回滚深度分位数随 RTT 的变化，虚线为常用缓冲窗口 $D_{\max} = 7$

在 RTT 100 ms 下，延迟式让玩家的每一次输入晚 6 帧（100 ms）才被游戏逻辑承认，而回滚式是 0 帧——这正是回滚同步的全部价值所在。差距随网络变差而扩大：RTT 200 ms 时延迟式要 8 帧，回滚式仍是 0 帧。图 4(b) 同时显示，回滚深度的分位数（平均深度、 p_{95} 、 p_{99} ）几乎与平均单向延迟同比例增长， p_{99} 在 RTT 200 ms 时达到 8 帧，刚好越过常用的 $D_{\max} = 7$ 缓冲窗口，于是出现停顿（第 5.5 节）。

5.4. 代价：额外模拟开销与可见翻案

表 6 与图 5 给出代价侧的结果（策略 S1，抖动 1 帧，丢包 2%，3 组随机种子平均）。

CPU 代价随单向延迟线性增长。额外模拟开销从 RTT 30 ms 的 17.1% 单调升到 200 ms 的 69.0%。对表 6 做线性回归可得

$$O \approx 11.3\% \times \text{owd} \text{ (帧)}, \quad (12)$$

即单向延迟每增加 1 帧，回滚就要多消耗约 11.3 个百分点的 CPU。这条经验公式的物理含义很直接： O 等于“每帧平均要重算的帧数”，而重算帧数 $\approx$ （回滚事件率） $\times$ （平均深度），事件率近似恒定（下一段），平均深度近似等于单向延迟。

表 6: 主实验结果：代价随 RTT 的变化（策略 S1, 200 s, 3 组种子平均）

RTT / ms	$\bar{D}$ / 帧	猜错率 / %	触发率 / %	p_{99} / 帧	最大深度 / 帧	CPU / %	可见翻案 / 次·min ⁻¹	停顿帧 / 200 s
30	1.33	15.9	15.3	3	3	17.1	0.4	0
60	1.88	18.6	16.0	4	4	22.6	0.7	0
90	2.66	22.1	15.7	4	5	31.0	4.1	0
120	3.54	25.5	15.0	5	6	41.0	14.5	0
150	4.43	28.7	14.4	6	7	51.4	24.4	0
200	5.94	33.4	14.1	8	8	69.0	31.4	38

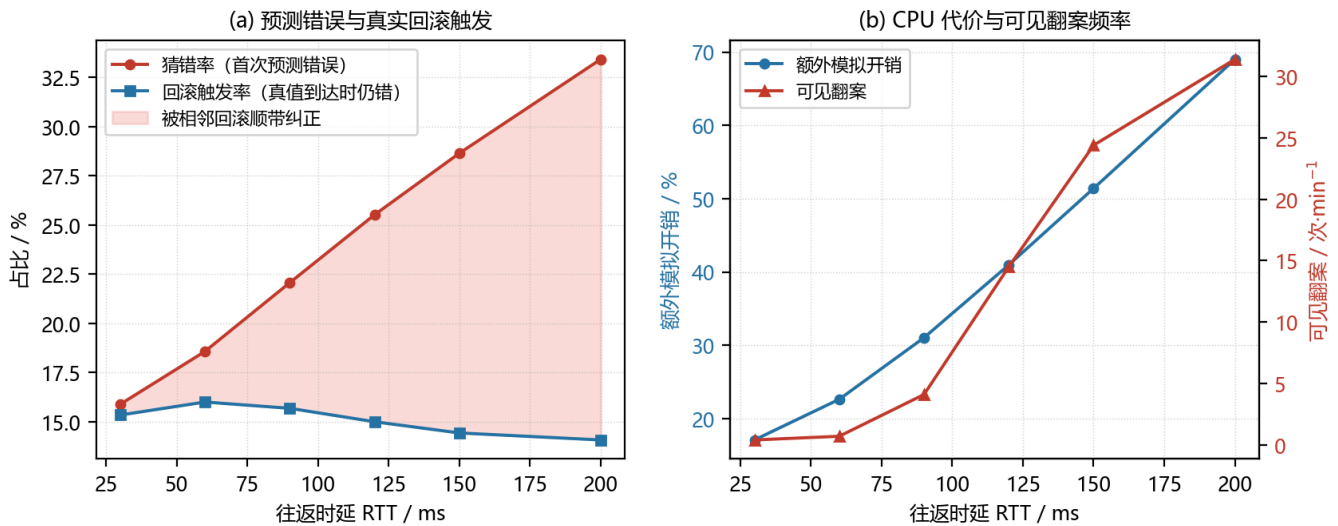


图 5: (a) 猜错率随 RTT 上升, 但真正触发回滚的比例几乎不变, 差额被相邻回滚顺带重算吸收; (b) CPU 额外开销随 RTT 近似线性增长, 可见翻案频率则呈超线性增长

猜错率与回滚触发率是两个不同的量。猜错率随 RTT 由 15.9% 升到 33.4%——这与图 3 的前瞻曲线高度一致（表 7），说明度量口径自治。但真正触发回滚的比例几乎不随 RTT 变化，稳定在 14.1%~16.0%。差额来自“顺带纠正”：被回滚区间内的帧会用更新的已知信息重新预测，其中相当一部分因此猜中，无需再触发一次回滚。顺带纠正率随 RTT 从 0.5% 升到 19.3%，恰好抵消了猜错率的上升。

表 7: 实测猜错率与前瞻曲线的交叉验证（策略 S1）

RTT / ms	30	60	90	120	200
平均单向延迟 / 帧	1.33	1.88	2.66	3.54	5.94
实测猜错率 / %	15.9	18.6	22.1	25.5	33.4
前瞻曲线预测值 / %	13.8	20.3	25.0	28.3	34.5

这一区分有实际意义：若按“猜错率”评估，会得出“延迟翻倍则回滚代价翻倍”的错误结论；实际上 CPU 代价只随深度线性增长，而事件率基本不变。

可见翻案是超线性的。可见翻案从 RTT 30 ms 的 0.4 次/分钟升到 200 ms 的 31.4 次/分钟，增速快于延迟本身。原因是翻案需要“预测错误”与“错误恰好落在命中判定的活动窗口内”同时成立，前者随延迟上升、后者随深度（暴露窗口长度）上升，二者相乘形成超线性。工程上的分水岭很清楚：RTT $\lesssim 60$ ms 时可见翻案低于 1 次/分钟，玩家基本察觉不到；超过 120 ms 后翻案升至 15 次/分钟以上，回滚的“副作用”开始变成可感知的体验问题。

预测策略对代价的影响很小。表 8 对比三种策略：在 RTT 30 ms 下 CPU 开销分别为 17.1%、17.1%、16.3%（S2 最好，因其猜错率最低 15.3%）；在 RTT 200 ms 下为 69.6%、69.0%、68.2%。三种策略的 CPU 开销相差不超过 2 个百分点，而网络条件带来的差异是 52 个百分点。结合第 5.2 节的结论，**提升预测器质量并不是降低回滚代价的有效手段——真正决定代价的是延迟本身。**

表 8: 三种预测策略的代价对比

策略	RTT 30 ms			RTT 200 ms		
	猜错率 / %	CPU / %	可见翻案	猜错率 / %	CPU / %	可见翻案
S0	16.20	17.1	0.4	34.32	69.6	34.4
S1	15.89	17.1	0.4	33.42	69.0	31.4
S2	15.32	16.3	0.4	33.69	68.2	32.8
S3（完美）	0	0	0	0	0	0

S3（完美预测）在所有配置下 CPU 开销与停顿均为 0，给出回滚代价的绝对下界：回滚的全部代价都来自预测错误，而预测错误在真实输入统计下无法被消除（第 5.2 节）。

5.5. 参数敏感性

图 6 与表 9 给出抖动、丢包、打法节奏与缓冲窗口四个方向的敏感性结果。

（策略 S2，除抖动与丢包两栏为 S0；猜错率一栏——表示该扫描不影响此量；缓冲窗口一栏的网络条件为 RTT 150 ms、抖动 2 帧、丢包 3%。）

抖动：代价下降，但深度尾部变长。抖动 σ 从 0 增到 4 帧时，CPU 开销反而从 40.85% 降到 28.62%，而最大回滚深度从 5 帧升到 11 帧、停顿累计 33 帧。这不是矛盾，而是式 (3) 的“取最小”在起作用：抖动增大时，冗余窗口内三个副本的最小到达时刻反而更接近当前帧，平均回滚深度下降（3.02→2.54 帧），因此 CPU 代价下降；但深度分布的尾部被拉长， $D_{\max}$ 会被打穿，停顿随之出现。结论是：**抖动主要威胁缓冲窗口的大小，而不是 CPU 预算。**同时，抖动把猜错率推高了约 4 个百分点（因为单帧的暴露窗口仍可能很长）。

丢包：3 重冗余几乎完全吸收。丢包率从 0 升到 5%，CPU 开销仅从 34.12% 升到 34.64%（变化 0.5 个百分点），200 s 内不可恢复的输入帧数从 0 增到 1。原因很直接： $R = 3$ 时一帧输入彻底丢失的概率是 p^3 ，5% 丢包下也只有 1.25×10^{-4} 。“每帧冗余重发最近 R 帧输入”这一看似朴素的工程手段，实际上是回滚同步鲁棒性的主要来源。

打法节奏是一阶影响因素。方向段平均长度 $\bar{L}$ 从 28 帧缩到 6 帧（方向切换率 7.8%→14.3%，对应从慢节奏试探到高压连段）时，猜错率从 15.74% 升到 32.57%，CPU 开销从 24.68% 升到 48.84%——几乎翻倍。这与网络延迟的影响（11.3 个百分点/帧单向延迟）是同一量级。也就是说，**回滚的代价既取决于网络，也取决于对局的激烈程度**，而后者在实测中常被忽略：同一对玩家，激烈对拼时服务器的 CPU 压力可能比平稳试探时高一倍。

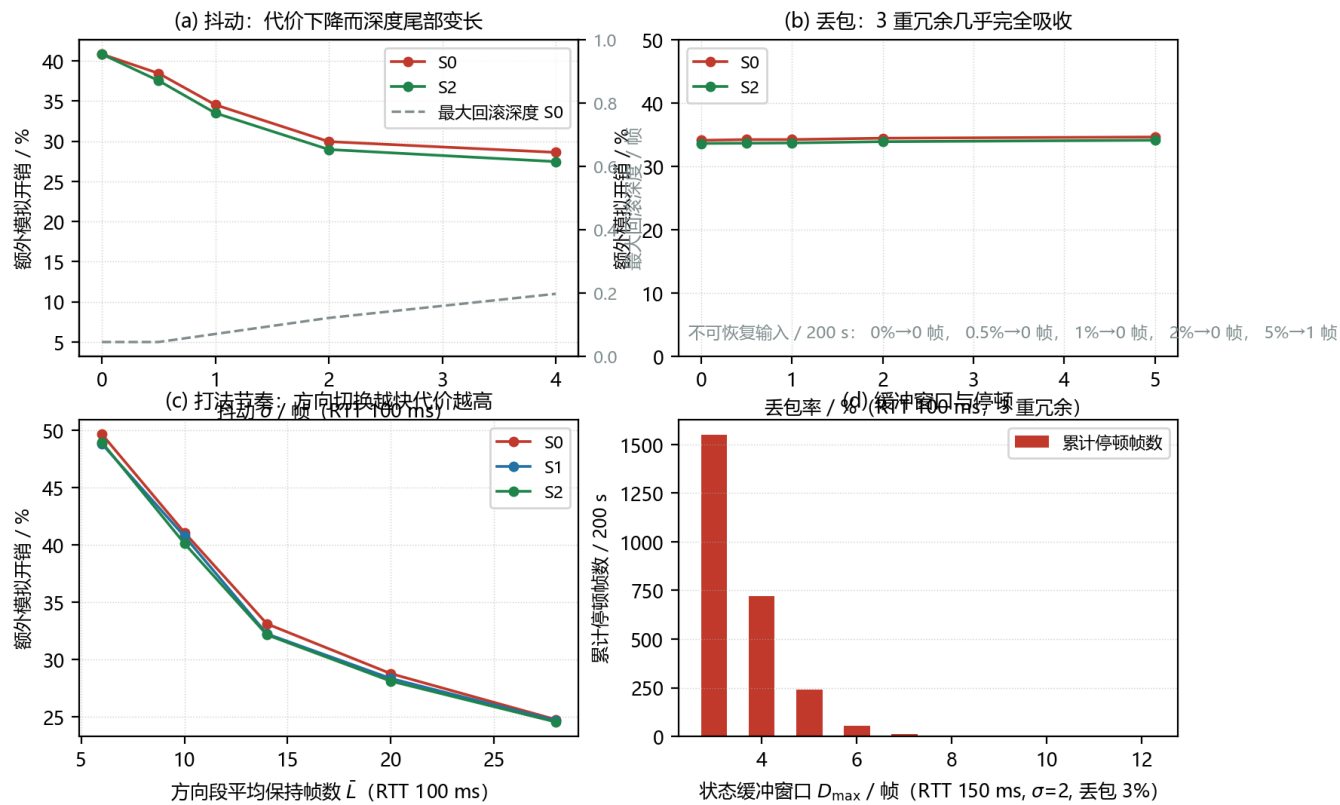


图 6: 参数敏感性: (a) 抖动使 CPU 代价下降但把深度尾部拉长; (b) 3 重冗余几乎完全吸收丢包; (c) 打法节奏是代价的一阶影响因素; (d) 缓冲窗口与停顿

表 9: 参数敏感性汇总 (RTT 100 ms, 除注明外)

扫描	取值	猜错率 / %	CPU / %	备注
抖动 σ / 帧	0	25.68	40.85	最大深度 5
	0.5	25.50	38.46	最大深度 5
	1	24.37	34.54	最大深度 6
	2	22.70	29.96	最大深度 8, 停顿 1 帧
	4	21.03	28.62	最大深度 11, 停顿 33 帧
丢包率 / %	0	—	34.12	不可恢复 0
	0.5	—	34.23	不可恢复 0
	1	—	34.24	不可恢复 0
	2	—	34.46	不可恢复 0
	5	—	34.64	不可恢复 1 帧
方向段长度 $\bar{L}$	6	32.57	48.84	切换率 14.3%
	10	28.09	40.77	切换率 12.0%
	14	22.61	32.24	切换率 10.2%
	20	18.73	28.35	切换率 8.8%
	28	15.74	24.68	切换率 7.8%
缓冲窗口 D_{max}	3	—	—	停顿 1550 帧
	5	—	—	停顿 241 帧
	7	—	—	停顿 10 帧
	10	—	—	停顿 0 帧

缓冲窗口与停顿。在 RTT 150 ms、 $\sigma=2$ 、丢包 3% 的较差网络下, $D_{\max}=3$ 会有 1550 帧的累计停顿(200 s 内相当于 12.9% 的时间在卡顿), $D_{\max}=7$ 降到 10 帧, $D_{\max}=10$ 即可完全消除停顿。由于该配置下最大回滚深度为 10 帧, $D_{\max}=10$ 是恰好够用的取值。缓冲窗口的代价是内存(每帧一份状态快照)与最坏情况下的重算量, 二者都远小于“卡顿”带来的体验损失, 因此工程上应取比典型值 7 更大的缓冲窗口。

5.6. 正确性验证

度量之所以可信, 前提是实现正确。本文在每次回滚仿真结束后, 把本地时间线与“全程使用真实远端输入”的真值时间线逐帧比较, 统计分歧帧数。表 6 的全部 6 个配置、表 8 的全部配置以及表 9 的全部配置, 分歧帧数均为 0: 即回滚机制在任意深度、任意抖动与丢包模式下都能把状态恢复到与真值完全相同的轨迹(位级一致)。唯一的例外是以极高丢包率造成“冗余窗口全部丢失、输入在本会话内不可恢复”的情形, 此时分歧帧数等于不可恢复帧数(实测最多 1 帧/200 s)。

这项验证同时反证了第 5.1 节的必要性: 如果内核使用浮点实现, 状态快照与重算就无法保证可复现, 本节的一致性将无从谈起。

六、讨论

6.1. 工程启示

综合上述结果, 回滚式同步的工程取舍可以归纳为四条:

(1) **确定性是前提, 不是优化。**定点数不仅“更稳”, 而是把同步从“概率问题”变成“确定问题”。式 (11) 一类的表达式必须显式写成“先乘后移位”, 并在构建系统里关闭 FMA 收缩; 否则分歧会在十几秒后静默出现。

(2) **延迟是代价的主导变量, 预测器不是。**CPU 开销约等于 $11.3\% \times$ 单向延迟(帧), 而三种预测策略的差异不超过 2 个百分点。把工程预算投在“降低 RTT”(就近服务器、更好的路由)上, 收益远大于投在“更聪明的预测器”上。

(3) **缓冲窗口应按网络的最坏情况而非典型值设计。** $D_{\max}=10$ 帧足以在 RTT 150 ms、抖动 2 帧、丢包 3% 下消除停顿, 而 7 帧会留下 10 帧停顿。缓冲窗口换的是内存与最坏情况重算量, 两者都远比卡顿廉价。

(4) **抖动对缓冲窗口的威胁大于对 CPU 的威胁。**抖动增大反而降低平均重算量(式 (3) 取最小), 但显著拉长深度分布的尾部。

6.2. 局限性

本文的结论受三个设定的限制。其一, 游戏内核是刻意极简的(一维场地、单一招式时间轴、无回合重置), 其绝对数值(如 $11.3\%/帧$ 的斜率)依赖单帧步进成本, 换一个更重的内核(复杂碰撞、AI、大量实体)会改变斜率, 但“代价与平均回滚深度成正比”的结构性关系不变。其二, 输入轨迹来自参数化生成模型而非真实玩家数据, 虽然四条统计规律与节奏扫描覆盖了较宽的范围, 但真实高手的搓招分布可能更集中、更长程相关, 这会改变预测器的相对优劣。其三, 本文只测量了客户端侧的重演代价, 未涉及带宽、NAT 穿透、帧同步校验等工程环节。

七、结论

本文用一套可复现的确定性仿真栈, 把回滚网络同步的“延迟—确定性—重演代价”三方权衡放到了同一组测量里: 用 Q16.16 定点数实现 60 Hz 双人格斗游戏内核, 以逐帧冗余链路模型驱动三种输入预测策略, 并与延迟式同步对照。主要结论是: (1) 仅 FMA 收缩一项就足以让 float64 实现在第 871 帧静默分歧, float32 第 1 帧即分歧, 而定点实现在语句重写后仍逐位一致; (2) 回滚把本地输入延迟从 3~8 帧降到 0 帧, 代价是 17.1%~69.0% 的额外模拟开销(约 11.3 个百分点/帧单向延迟)和 0.4~31.4 次/分钟的可见翻案; (3) 猜错率随延迟由 15.9% 升到 33.4%, 但真正触发回滚的比例稳定在 14.1%~16.0%, 差额被相邻回滚顺带重算吸收——“预测错误”与“回滚代价”必须分开度量; (4) 预测增益上限很小, 方向肢已被“保持”策略封顶(1 帧前瞻 84.33%), 三种策略的 CPU 代价相差不到 2 个百分点, 而网络条件带来的差异是 52 个百分点; (5) 3 重冗余可把 5% 丢包的影响压到 0.5 个百分点以内, $D_{\max}=10$ 帧即可消除 RTT 150 ms、抖动 2 帧、丢包 3% 下的停顿; (6) 所有回滚时间线与真值逐位一致, 验证了实现与度量口径的正确性。

参考文献

- [1] copycat666, doro, and mysteriousK. “从“云”到“边缘”——电子游戏分布式计算架构的技术发展综述”. In: 电子游戏科学与技术 3 (2026). DOI: [ET-3-2-2026-R](#).
- [2] Paul Bettner and Mark Terrano. “1500 Archers on a 28.8: Network Programming in Age of Empires and Beyond”. In: *Proceedings of the Game Developers Conference (GDC)*. San Jose, CA, 2001.
- [3] twinkHowdyLittleDark et al. “辛积分器在实时游戏物理中的节能性研究——以平方反比引力场射击游戏为实验平台”. In: 电子游戏科学与技术 3 (2026). DOI: [ET-3-9-2026-R](#).
- [4] IEEE. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019. 2019. DOI: [10.1109/IEEESTD.2019.8766229](#).
- [5] David Goldberg. “What Every Computer Scientist Should Know About Floating-Point Arithmetic”. In: *ACM Computing Surveys* 23.1 (1991), pp. 5–48. DOI: [10.1145/103162.103163](#).
- [6] Jean-Michel Muller et al. *Handbook of Floating-Point Arithmetic*. 2nd ed. Birkhäuser, 2018. DOI: [10.1007/978-3-319-76526-6](#).
- [7] David Monniaux. “The Pitfalls of Verifying Floating-Point Computations”. In: *ACM Transactions on Programming Languages and Systems* 30.3 (2008), 12:1–12:41. DOI: [10.1145/1353445.1353446](#).
- [8] Tony Cannon. *GGPO: Fast, Good-Enough Rollback Networking*. 开源软件库, <https://github.com/pond3r/ggpo>. 2009.
- [9] Yahn W. Bernier. “Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization”. In: *Proceedings of the Game Developers Conference (GDC)*. 2001.
- [10] Gabriel Gambetta. *Fast-Paced Multiplayer*. 技术专栏, <https://gabrielgambetta.com>. 2018.
- [11] Glenn Fiedler. *Networked Physics*. Gaffer On Games 技术专栏, <https://gafferongames.com>. 2014.
- [12] John G. Cleary and Ian H. Witten. “Data Compression Using Adaptive Coding and Partial String Matching”. In: *IEEE Transactions on Communications* 32.4 (1984), pp. 396–402. DOI: [10.1109/TCOM.1984.1096090](#).
- [13] Christopher D. Manning and Hinrich Schütze. *Foundations of Statistical Natural Language Processing*. Cambridge, MA: MIT Press, 1999.